First Automation FIN1209: ask R whether to buy or sell

Write a short R script that reads a stock's prices, takes five votes a day (price trend, price bar, RSI, MACD and stochastic), and answers: buy today or wait, or, if you own it, sell now or hold. Then draw a flowchart that explains it.

Student worksheet 100 points | a few hours over a week

Benjamin C. Sotelo | Institute of Accounts, Business and Finance, FEU Manila

Name _____ Section _____ Date _____

What you are making

Read this page first. It is short.

You will teach R to answer one question for a stock you choose: **should I buy it today, or, if I already own it, should I sell it?** You put a Google Sheets price file in a folder, type four settings at the top of a short **R script**, and run it. It reads **price, bars and volume** (Chapters 1 to 7) and the three **window oscillators** of Chapter 8, takes five votes for buy or sell every day, and keeps the days when **at least 3 of the 5 agree**. In **buy mode** it says BUY, with a level to enter above and a stop, or WAIT. In **sell mode**, from the price and the day you bought, it says SELL now or HOLD, with a stop. It writes the answer, a table and a chart. Add new prices and run it again, and the answer updates. That is **automation**.

> THIS PROGRAM IS NOT FOOLPROOF
>
> **It follows rules from our slides. Its signals can be wrong, and they can lose money. It is a class exercise, not advice.**

> YOU DO NOT NEED TO KNOW R
>
> Nobody in this class was taught to code. The script is about 90 lines in six labelled blocks. Part 5 walks through it block by block: each line shown whole and highlighted, then its key part, with one plain sentence on each. The script uses no pipe operators, only plain R and one package.

What you get, and what you hand in

	WHAT
You get	This manual, a guide video, the finished example script and the example flowchart (`.pdf` and `.bpmn`). No price files: you download them yourself.
You hand in	Your own R script, built on your own rules, not a copy of the example with the stock changed (Part 9), with the four settings lines at the top, and a flowchart that also shows where your script differs from the example. Your own stock. Share a Drive folder, as Part 9 describes.
Due date	Sunday, 15 November 2026, 11:59 PM.

> THIS ALSO COUNTS AS YOUR CHAPTER 8 QUIZ AND YOUR FINAL QUIZ
>
> **There will be no reschedules for this activity: late or missing work is not re-opened.** This is too hard to check case by case, so there is no exception.

WHAT THE ANSWER IS, AND IS NOT

The answer is what the rules say about past prices **up to the last row of your file**. That row is the latest trading day in your download, not the live market price: Google Sheets prices can be up to about 20 minutes late, and the script does not read a live price. It does not say what price will do next. Chapter 1's point still holds: a forecast cannot be checked when it is made.

1. What to install

Free, and about 30 minutes. If you cannot install, use the browser option.

You need two programs. **R** is the language, and it does the work. **RStudio Desktop** is a friendly window for writing and running R. Install R first, then RStudio.

> ONE PACKAGE: TTR
>
> The example uses one package, **TTR**, which works out the moving average, the RSI, the MACD and the stochastic. Install it once, in the Console, by typing `install.packages("TTR")`. Never put `install.packages` inside your script. Use no other package: the marker's computer has TTR and nothing else.

W Windows

1. Go to **https://cran.r-project.org/bin/windows/base/** and click the **Download R** link at the top. Open the file you downloaded and click Next through the installer, keeping every default.
2. Go to **https://docs.posit.co/ide/user/download.html** and click the big **Download for Windows** button. Open the file and click Next through the installer.
3. Open **RStudio** from the Start menu. A window opens with a pane called Console on the left. That is where R talks.

M Mac

1. Go to **https://cran.r-project.org/bin/macosx/**. Click the first `.pkg` under the latest release, the one for your chip (the Apple icon, then About This Mac, tells you whether it is Apple silicon or Intel). Open the file and click Continue through the installer.
2. Go to **https://docs.posit.co/ide/user/download.html**, click **Download for macOS**, open the file, and drag RStudio into the Applications folder.
3. Open **RStudio** from Applications. A window opens with a pane called Console on the left.

C If you cannot install anything

Use **Posit Cloud**, which runs RStudio in your browser. Go to **https://posit.cloud**, make a free account, then **New Project**, then **New RStudio Project**. Everything in this handout works the same way. To put the example folder in, use the **Upload** button in the Files pane.

F For the flowchart

Draw it in **bpmn.io** at **https://demo.bpmn.io/**. It runs in the browser, is free, and needs no account. Or install **Camunda Modeler** from **https://camunda.com/download/modeler/**. Both draw BPMN and both save a `.bpmn` file and export a picture. If you would rather use Markdown, **https://mermaid.live** draws a Mermaid flowchart in the browser.

2. Where to get price data, and what kind

Checked on the public sites on 4 November 2026. Where a claim needs an account, it says so instead of guessing.

Chapter 1 names the data technical analysis reads: **price, volume and open interest**. For this activity you need one bar per trading day, with six columns.

The script needs six columns: Date, Open, High, Low, Close and Volume. The RSI and the MACD use the close; the stochastic uses the high, low and close; a buy's trigger uses the high and a sell's the low. Volume marks a signal CONFIRMED when it is above the file's average. **Open interest** exists only for futures, so a stock has none, and no source below gives one for a share.

The free sources, and what we found

SOURCE	OPEN, HIGH, LOW, CLOSE, VOLUME	ACCOUNT	CSV FILE
Google Sheets, GOOGLEFINANCE	Yes, daily or weekly, all five, with the formula `"all"`. The Help page lists them.	A Google account	Yes: File, Download, CSV. **This handout uses it, for US-listed stocks only.** `PSE:JFC` returned `#N/A`, so there is no daily history for Philippine stocks.
PSE EDGE (edge.pse.com.ph), the exchange's own site	Today's open, high, low, last price and volume, and the 52-week range, for each Philippine stock. We found no table of past days.	None to read	No download link found. The site says its content is copyrighted.
Yahoo Finance	A history table of date, open, high, low, close, adjusted close and volume for US stocks, readable without signing in. We found no Philippine stocks.	None to read the table	No Download button when signed out. Whether one appears after signing in is not checked.
Investing.com	Not checked: the site refused our automated visit. It is a common place to look for Philippine stocks, so check what it offers for the one you want.	Not checked	Not checked
TradingView	Charts Philippine stocks, for example PSE:BDO, with a free public page. Export of the chart data was not checked.	None to view; export needs an account	Not checked, needs an account
Your broker's platform, for example COL Financial	Not checked: it sits behind a login.	Yes, a brokerage account	Not checked

Prices in the **adjusted close** column are corrected for splits and dividends. The example does not use it.

The script works with any source whose CSV has the columns of Part 5. Google Sheets is the one route this handout walks through step by step, because every student already has an account and it exports a CSV in the right layout. If you find Philippine data elsewhere and can save it as a CSV with those column names, it will run.

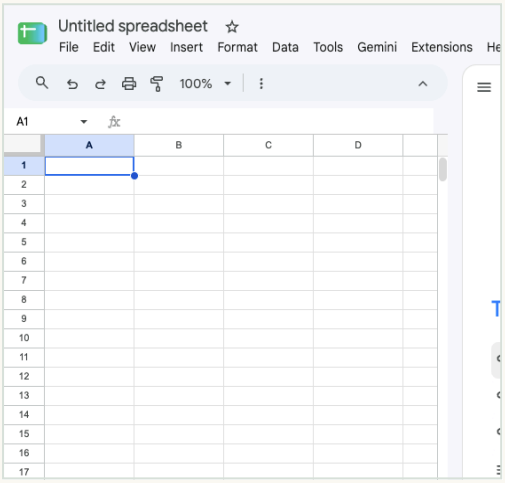
2b. Get the example prices yourself

The same route you will use for your own task. We give you no price files.

Google Sheets can fetch daily prices with one formula, and it gives exactly the columns the contract needs: Date, Open, High, Low, Close, Volume. It works for **US-listed stocks**. You do this once per stock, and you use the same dates as below so your numbers match this handout.

1 Open a blank spreadsheet

Go to **https://sheets.new** while signed in to your Google account. A blank spreadsheet opens. Click cell **A1**.



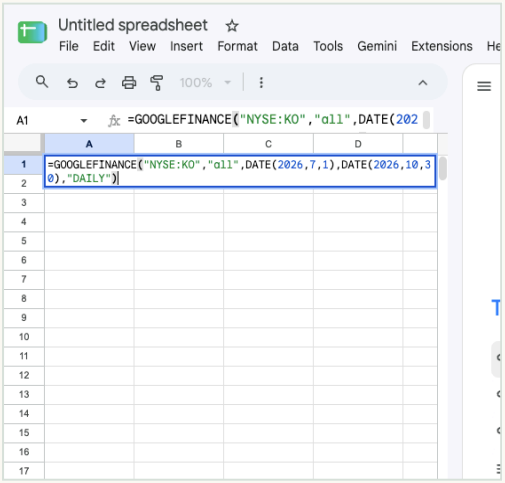
2 Type the formula

In A1 type this formula for Coca-Cola, then press Enter:

TYPE EXACTLY THIS

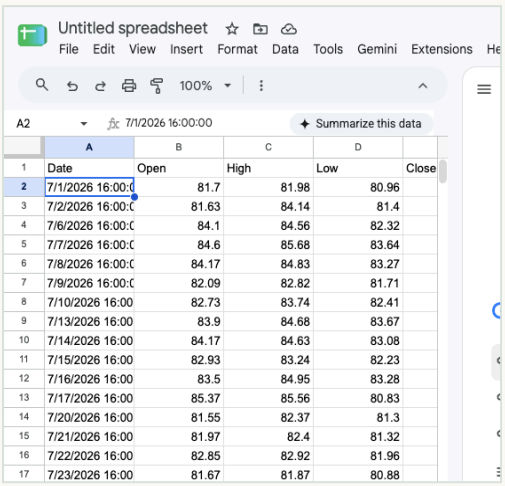
```
=GOOGLEFINANCE("NYSE:KO","all",DATE(2026,7,1),DATE(2026,9,30),"DAILY")
```

The picture shows the formula being typed. Its end date in that capture reads 2026,10,30; **use 2026,9,30** for this first run.



3 Check the result

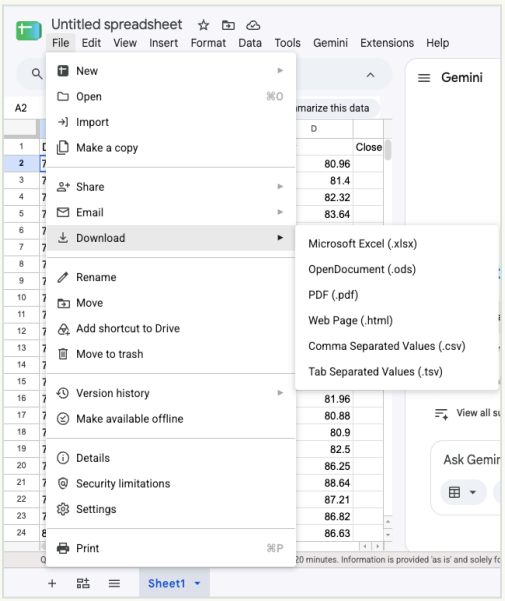
Wait a moment. About 64 rows fill down from A1, with the six column names in row 1. The dates read like `7/1/2026 16:00:00`: that is the layout Google Sheets gives, and the script reads it as it is, so you never edit a date. If you see `#N/A`, check the quote marks are straight and the ticker is spelled `NYSE:KO`. The last row is the latest trading day, and Google prices can be up to about 20 minutes late.



	A	B	C	D	
1	Date	Open	High	Low	Close
2	7/1/2026 16:00:0	81.7	81.98	80.96	
3	7/2/2026 16:00:0	81.63	84.14	81.4	
4	7/6/2026 16:00:0	84.1	84.56	82.32	
5	7/7/2026 16:00:0	84.6	85.68	83.64	
6	7/8/2026 16:00:0	84.17	84.83	83.27	
7	7/9/2026 16:00:0	82.09	82.82	81.71	
8	7/10/2026 16:00	82.73	83.74	82.41	
9	7/13/2026 16:00	83.9	84.68	83.67	
10	7/14/2026 16:00	84.17	84.63	83.08	
11	7/15/2026 16:00	82.93	83.24	82.23	
12	7/16/2026 16:00	83.5	84.95	83.28	
13	7/17/2026 16:00	85.37	85.56	80.83	
14	7/20/2026 16:00	81.55	82.37	81.3	
15	7/21/2026 16:00	81.97	82.4	81.32	
16	7/22/2026 16:00	82.85	82.92	81.96	
17	7/23/2026 16:00	81.67	81.87	80.88	

4 Download it as a CSV

Choose **File, Download, Comma Separated Values (.csv)**. The file holds the values, not the formula, so there is nothing more to do to it. Your browser saves a file named after the spreadsheet.



Rename the file **KO.csv** and move it into your `data` folder. The name matters: the script takes the ticker from it. Then do the same for a second stock in a new spreadsheet, with this formula, and save it as **F.csv** in `data`. Part 6 runs the script on these two files.

CELL A1, FOR F (FORD)

```
=GOOGLEFINANCE("NYSE:F","all",DATE(2026,7,1),DATE(2026,9,30),"DAILY")
```

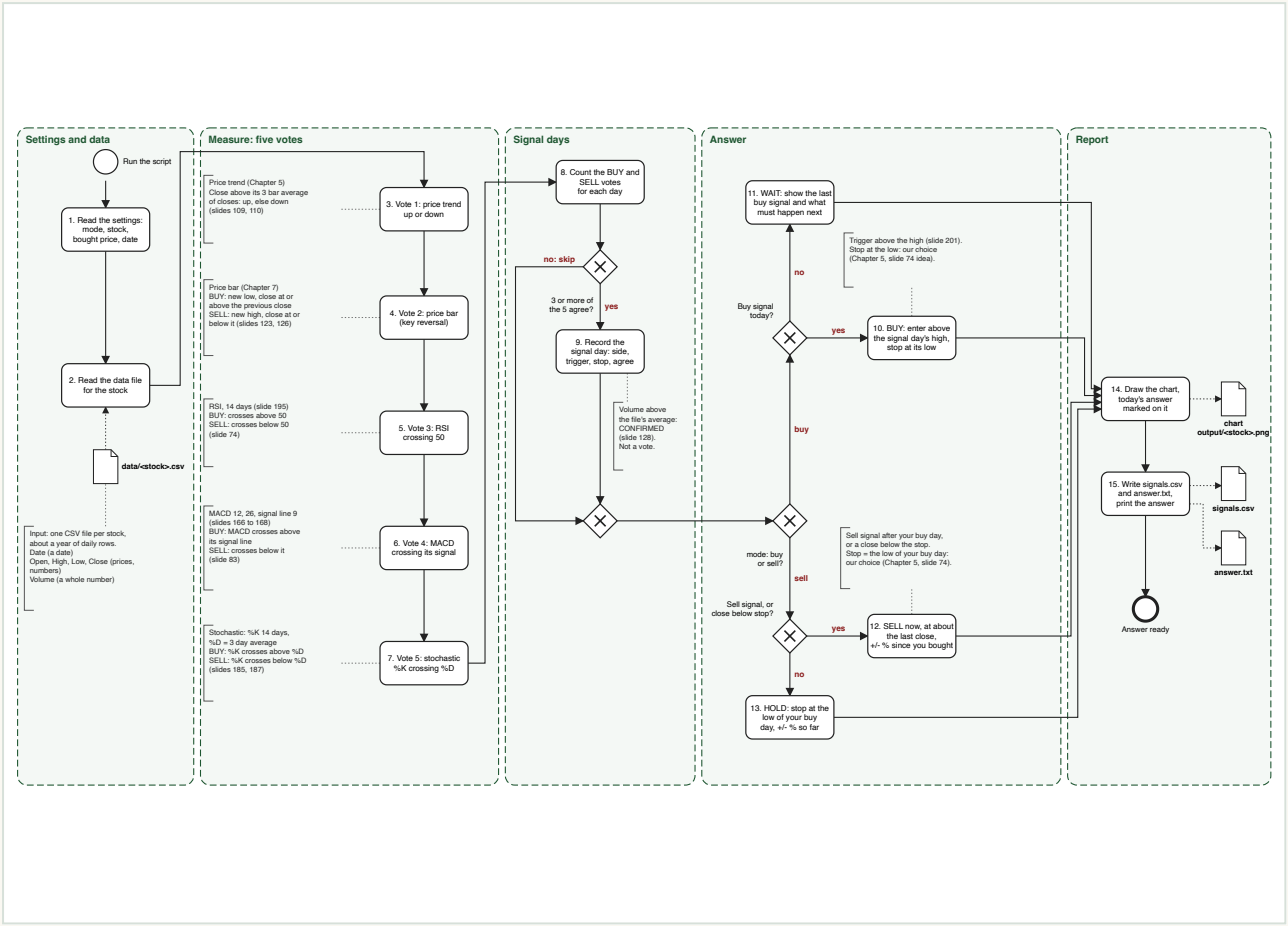
Later, for the update in Part 6: in each spreadsheet change `DATE(2026,9,30)` in the formula to `DATE(2026,10,30)`, download again, and replace the file in `data`, still named KO.csv and F.csv. There is no capture of that second download because it is the same four steps.

SWAP THE TICKER, NOTHING ELSE

To use another US stock, change only the text inside the first quotes of the formula, for example `"NASDAQ:AMD"`. **Philippine stocks do not work here.** We tried `PSE:JFC` and Sheets answered `#N/A`: GOOGLEFINANCE gives no daily history for Philippine stocks. For a PSE stock you need another source from Part 2, for example Investing.com, then save its data as a CSV with the columns of Part 5.

3. What the script does

Input, process, output. A flowchart says it in shapes; yours must match your script line for line.



The example script as a BPMN flowchart. `First Automation FIN1209 - Example Flowchart.bpmn` opens in bpmn.io.

Drawn for this course by build/bpmnkit.py.

Input, process, output

PART	WHAT HAPPENS
Input	Four settings you type (buy or sell, your stock, and for a sale the price and day you bought), and a price file from Google Sheets: a date, four prices and a volume for each trading day.
Process	Five votes for buy or sell every day: the price trend and the price bar (Chapters 5 and 7), then the RSI, the MACD and the stochastic crossing (Chapter 8). A signal day is a day when at least 3 of the 5 agree. Then your mode decides: BUY or WAIT, or SELL now or HOLD.
Output	The answer, in the Console and in `answer.txt`; a table of the signal days; a chart of price, volume and the three oscillators with today's answer marked.

Which lines each shape stands for

SHAPE	SAYS	LINES
Start event	Run the script	the moment you click Source
1	Read the settings: mode, stock, bought price, bought date	lines 3 to 6
2	Read the data file for the stock	lines 9 to 11
3	Vote 1: price trend	lines 21 to 23
4	Vote 2: price bar (key reversal)	lines 24 to 26
5	Vote 3: RSI crossing 50	line 27 and line 30
6	Vote 4: MACD crossing its signal line	line 28 and line 31
7	Vote 5: stochastic %K crossing %D	line 29 and line 32
8	Count the BUY and SELL votes for each day	lines 33 to 36
Gateway	3 or more of the 5 votes agree?	line 37
9	Record the signal day: side, trigger, stop, agree, volume	lines 38 to 45
Gateway	Mode: buy or sell?	line 46
Gateway	Buy signal today?	line 48
10	BUY: enter above the signal day's high, stop at its low	line 49
11	WAIT: the last buy signal and what must happen next	lines 51 to 54
Gateway	Sell signal since you bought, or close below the stop?	line 63
12	SELL now, at about the last close, with the percent	line 64
13	HOLD: your stop and the percent so far	line 68
14	Draw the chart with today's answer marked	lines 73 to 87
15	Write signals.csv and answer.txt, print the answer	lines 89 to 91

BPMN needs five shapes: a thin circle starts, a thick circle ends, a rounded rectangle is a task named with a verb, a diamond with an X is a yes or no question with every arrow labelled, and an arrow is the order. A page-shaped data object is a file, and a text annotation says a rule. Every shape on your flowchart should point at lines of your script.

4. The five votes and the volume check

Each rule as the slides give it, with the slide number and title. Chapter 5, 7 and 8 student editions.

What the slides say

SLIDE	RULE
Ch 5: 109, 110 "Drummond geometry", "Reading a PLdot line"	**Trend vote.** A three bar moving average of price: price above it is bullish, below it is bearish.
Ch 7: 123, 126 "The key reversal is when a strong move suddenly turns"	**Bar vote.** A bearish key reversal bar forms a higher high than the previous bar and closes near or below the previous close. The bullish one is the opposite: a new low, then a close near or above.
Ch 7: 128 "Figure 7.15 shows the key reversal on the book's chart"	**Volume check.** The key reversal day is marked with above-average volume. Volume is not a vote: it marks a signal CONFIRMED or not.
Ch 8: 74	**RSI vote.** Crossing above the 50 equilibrium line is a buy and below it a sell.
Ch 8: 83, 166 to 168	**MACD vote.** 12 period EMA minus 26 period EMA, 9 period signal line. Crossing above the signal line is a buy, below it a sell.
Ch 8: 185, 187, 189	**Stochastic vote.** %K over 14 bars, %D its 3 period average. %K crossing above %D is a buy, below it a sell.
Ch 8: 193 to 195	The RSI uses 14 periods and plain averages of gains and losses.
Ch 8: 201	The trigger for entry is normally the high of the bar tied to the buy signal.
Ch 8: 78, 81	Oscillators built from the same prices are highly correlated: multicollinearity, a false consensus.

OUR CHOICES, NOT LIM'S RULE

Trend. Slide 110's own chart averages closes, and the book's PLdot averages typical price, which Chapter 5 does not define. We average the closes. Chapter 2's peaks and troughs trend is not computed.

Key reversal. The slides say "near or below" the previous close. We read it as at or below, and at or above for the bullish one.

Volume. The slides give no period for "average", so it is the average volume of the whole file.

Entry. A buy triggers above the signal day's high, as slide 201 says. The slides give no sell trigger, so a sell triggers below the signal day's low: the mirror of Lim's buy trigger on slide 201, our choice, not Lim's rule. (Lim's own drawing of the ID/NR6 pattern, Chapter 7 slide 207, puts its long trigger at a bar's high and its short trigger at that bar's low, but that is one pattern.) For both sides the price is a trigger level: the trade happens only if price later goes beyond it.

Signal day. The slides do not say how many votes must agree. At least 3 of the 5 is our choice. `agree` counts the votes on the winning side.

Your stop. Chapter 5, slide 74 says a stop is usually placed just below a significant trough for a long position, and gives no rule or number. Ours is the low of the signal day in buy mode, and the low of the day you bought in sell mode. It is our choice, not the book's rule.

What the script answers

ANSWER	WHEN, AND WHAT IT PRINTS
Signal day	A day when at least 3 of the 5 votes agree: 3 or more BUY, or 3 or more SELL. `agree` counts them.
Today	The last row of your file: the latest trading day in the download. It is not a live price; Google Sheets prices can be up to about 20 minutes late.
BUY (buy mode)	Today is a buy signal day. Enter if price trades above today's high (slide 201), with a stop at today's low.
WAIT (buy mode)	Today is not a buy signal day. It says how many votes say BUY today, shows the last buy signal in the file with its levels, and says what must happen next: 3 of the 5 votes on BUY.
SELL now (sell mode)	Since the day you bought there was a sell signal, or the price closed below your stop. It prints about the last close and your gain or loss in percent, and why.
HOLD (sell mode)	Neither happened. It prints your stop and your gain or loss so far.

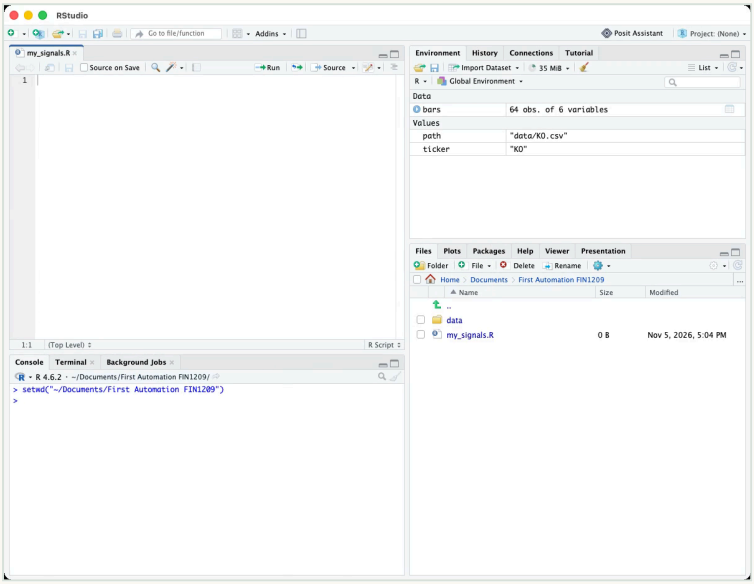
In sell mode you give the script the price you paid and the day you bought, in the four settings lines. In buy mode it assumes today's price is the last row of your file.

5. The script, block by block

Six labelled blocks. Each important line is shown whole, then its key part, with one plain sentence each.

1 Set up

Make a folder `First Automation FIN1209` in your Documents folder with a folder `data` inside it, and put the two Part 2 files `KO.csv` and `F.csv` in `data`. In RStudio choose **File, New File, R Script**, save it in the folder, then **Session, Set Working Directory, To Source File Location**. Type `install.packages("TTR")` in the Console once. Run your script with **Code, Source**.



The example script, `First Automation FIN1209 - Example Script.R`, has six labelled blocks. Read them in order. Each line below is shown whole on a yellow bar, then the key part of it is boxed, with one plain sentence on each. The script uses **no pipe operators**: every line is plain R.

1 Settings

Block 1, lines 2 to 6. The only lines you edit. Change a value, run the script, read the answer.

```
3  mode <- "buy"                  # "buy" or "sell"
```

This line: Your question. "buy" asks: should I buy today? "sell" asks: I already own it, should I sell?

The boxed part: The word in quotes is the setting. Type buy or sell exactly, in small letters.

```
4  stock <- "KO"                  # which file in the data folder: data/KO.csv
```

This line: Which price file to read. This one reads data/KO.csv.

The boxed part: Your stock's name. It must match the file name in the data folder.

```
5  bought_price <- 87.29          # sell mode only: the price you paid
```

This line: Sell mode only: the price you paid for each share.

The boxed part: A plain number, with no quotes.

```
6  bought_date <- "2026-08-12"    # sell mode only: the day you bought it
```

This line: Sell mode only: the day you bought it.

The boxed part: Year, month, day, in quotes. It must be a trading day in your file.

2 Read the data

Block 2, lines 7 to 11. Loads the indicator package and your stock's prices.

```
 8 library(TTR)
```
This line: Loads the package that works out the moving average, the RSI, the MACD and the stochastic.

The boxed part: TTR is the only package we use. Install it once, in the Console, with install.packages("TTR").

```
 9 bars <- read.csv(file.path("data", paste0(stock, ".csv")))
```
This line: Reads your stock's file into a table called bars, one row for each trading day.

The boxed part: paste0 glues pieces of text together: the stock KO and .csv make KO.csv.

```
10 bars$Date <- as.Date(sub(" .*", "", bars$Date), tryFormats = c("%Y-%m-%d", "%m/%d/%Y"))
```
This line: Turns the Google Sheets dates into real dates, so R can sort and plot them.

The boxed part: Sheets writes a time after the date. This cuts off everything from the first space.

```
11 n <- nrow(bars)                # the last row is the latest trading day in the file: "today"
```
This line: n is the number of rows. Row n, the last row, is the latest trading day in the file: "today".

The boxed part: Today is the last day in your download, not a live price. Sheets prices can be about 20 minutes late.

3 The five votes

Block 3, lines 12 to 33. Each vote is +1 for BUY, -1 for SELL or 0 for nothing.

```
13 before <- function(x) c(NA, head(x, -1))      # yesterday's value, in today's row
```
This line: A small helper: yesterday's value, placed in today's row.

The boxed part: Head with minus one is the list without its last value. The NA in front moves everything down one day.

```
14 cross <- function(a, b) {      # +1 on the day a crosses above b, -1 on the day it crosses
   below
```
This line: A helper that spots a crossing: +1 the day line a crosses above line b, -1 the day it crosses below, 0 otherwise.

The boxed part: A function packs lines under a name, so we write them once and use them three times.

```
17   s <- ifelse(a > b & before(a) <= before(b), 1, ifelse(a < b & before(a) >= before(b), -1, 0))
```
This line: Above b today but not above b yesterday: that is a crossing up.

The boxed part: ifelse picks a value for every day at once: the first value when the test is true, the second when it is false. The ampersand means and.

```
21 pldot <- SMA(bars$Close, 3)
```
This line: The three day average of the closes: the PLdot line of Chapter 5 (slides 109, 110).

The boxed part: SMA is a simple moving average: the average of the last 3 closes.

```
22 trend_vote <- ifelse(bars$Close > pldot, 1, -1)
```
This line: Vote 1, the price trend: +1 when the close is above its average, -1 when it is below.

The boxed part: Greater than: it asks whether the close is bigger than its average.

```
24 bar_vote <- ifelse(bars$Low < before(bars$Low) & bars$Close >= before(bars$Close), 1,
```
This line: Vote 2, the price bar, a key reversal (Chapter 7, slides 123, 126): a lower low that closes at or above yesterday's close votes BUY; a higher high that closes at or below it votes SELL.

The boxed part: The ampersand joins two conditions: both must be true on the same day.

```
27 rsi <- RSI(bars$Close, n = 14, maType = "SMA")
```
This line: The RSI over 14 days (Chapter 8, slides 193 to 195).

The boxed part: n = 14 is how many days it looks back.

```
28 macd <- MACD(bars$Close, nFast = 12, nSlow = 26, nSig = 9, percent = FALSE)
```
This line: The MACD: two averages (12 and 26 days) and a 9 day signal line (slides 166 to 168).

The boxed part: These numbers are the settings the slides use.

```
29 sto <- stoch(bars[, c("High", "Low", "Close")], nFastK = 14, nFastD = 3)
```
This line: The stochastic: %K over 14 days and %D, its 3 day average (slides 185, 187).

The boxed part: It needs the High, Low and Close columns, so we hand it those three.

```
30 rsi_sig <- cross(rsi, 50)
```
This line: Votes 3 to 5: +1 when the RSI crosses above 50 and -1 below (slide 74). The next two lines do the same for the MACD and its signal line (slide 83) and for %K and %D (slide 187).

The boxed part: The three lines use the same cross helper from above.

```
33 votes <- cbind(trend_vote, bar_vote, rsi_sig, macd_sig, stoch_sig)
```
This line: Puts the five votes side by side: one column each, one row for each day.

The boxed part: cbind binds columns together.

4 Decide: signal days, then buy or sell

Block 4, lines 34 to 70. Counts the votes, finds the signal days, then your setting decides what to answer.

```
35 buys <- rowSums(votes == 1)
```
This line: For each day, counts how many of the five votes say BUY. The next line counts SELL.

The boxed part: Where a vote says BUY the test is true. rowSums adds up the true values across the five votes.

```
37 day <- which(buys >= 3 | sells >= 3)
```
This line: A signal day is a day when at least 3 of the 5 votes agree. which() gives the row numbers of those days.

The boxed part: The vertical bar means or: 3 or more BUY, or 3 or more SELL.

```
40 signals <- data.frame(ticker = rep(stock, length(day)), date = bars$Date[day], side = side,
```
This line: Builds the table of signal days, one row for each signal.

The boxed part: data.frame makes a table from columns of equal length.

```
41                         entry = ifelse(side == "BUY", bars$High[day], bars$Low[day]),
```
This line: The trigger level. A buy triggers above the signal day's high (slide 201). A sell triggers below its low: the mirror, our choice. The trade happens only if price later goes beyond the level.

The boxed part: Square brackets pick rows: this is the High column, on the signal days only.

```
46 if (mode == "buy") {           # BUY: the latest day is a buy signal. WAIT: it is not.
```
This line: Now your setting decides. Everything in the first brackets is buy mode.

The boxed part: The double equals sign asks: is it equal to?

```
48   if (n %in% day[side == "BUY"]) {
```
This line: Is today, row n, one of the buy signal days?

The boxed part: %in% asks whether a value is in a list.

```
49     answer <- paste0("BUY: enter if price trades above ", bars$High[n], ", stop at ",
   bars$Low[n])
```
This line: BUY: enter if price trades above today's high, with the stop at today's low.

The boxed part: Square brackets pick row n, today.

```
51     answer <- paste0("WAIT: no buy signal today, ", buys[n], " of 5 votes say BUY and 3 are
   needed")
```
This line: WAIT: no buy signal today. It says how many votes say BUY today and that 3 are needed.

The boxed part: Square brackets again: the count for today.

```
57   i <- max(which(bars$Date <= as.Date(bought_date)))
```
This line: Sell mode starts here: finds the row of the day you bought.

The boxed part: Max of which takes the last row that is on or before your date.

```
58    stop_at <- bars$Low[i]        # our choice: the low of the day you bought (Chapter 5, slide
    74: a stop sits below a trough)
```
This line: The stop: the low of the day you bought. Our choice. Chapter 5, slide 74 says a stop sits just below a significant trough but gives no rule.

The boxed part: i is the row of your buy day.

```
60    sell_day <- day[side == "SELL" & day > i]
```
This line: The sell signal days after the day you bought.

The boxed part: Only days after your buy day count.

```
61    below <- later[bars$Close[later] < stop_at]
```
This line: The days after you bought that closed below your stop.

The boxed part: Less than: it asks whether the close is smaller than your stop.

```
62    pct <- sprintf("%+.1f%%", (bars$Close[n] / bought_price - 1) * 100)
```
This line: Your gain or loss so far, in percent, with a plus or minus sign.

The boxed part: sprintf turns a number into text in the layout you choose.

```
63    if (length(sell_day) > 0 || length(below) > 0) {
```
This line: SELL now if there was a sell signal or a close below the stop. Otherwise the answer is HOLD.

The boxed part: The double bar means or.

5 Draw the chart

Block 5, lines 71 to 87. One picture: price, volume and the three oscillators, with today's answer marked.

```
73  png(file.path("output", paste0(stock, ".png")), width = 1200, height = 1200)
```
This line: Opens a picture file in the output folder, named after your stock.

The boxed part: The size of the picture in pixels.

```
76  plot(bars$Date, bars$Close, type = "l", xlab = "", ylab = "Close", main = paste(stock, "-",
    answer[1]))
```
This line: Draws the closing price against the date. The title carries today's answer.

The boxed part: type = "l" means a line.

```
78  points(signals$date, signals$entry, pch = ifelse(side == "BUY", 24, 25), bg = ifelse(side ==
    "BUY", "darkgreen", "red"), cex = 1.5)
```
This line: A triangle on every signal day: green pointing up for a buy, red pointing down for a sell.

The boxed part: pch is the shape of the mark: 24 points up, 25 points down.

```
79  abline(v = bars$Date[n], lty = 2)
```
This line: Today's answer on the chart: a dashed line at the latest day, labelled BUY, WAIT, SELL or HOLD.

The boxed part: abline draws a straight line. v is for vertical.

```
81  if (mode == "sell") { points(bars$Date[i], bought_price, pch = 4, cex = 2, lwd = 3); abline(h =
    stop_at, lty = 3, col = "red") }
```
This line: In sell mode only: a cross where you bought and a dotted red line at your stop.

The boxed part: The cross is drawn at your buy date and the price you paid.

```
82  plot(bars$Date, bars$Volume, type = "h", xlab = "", ylab = "Volume", col = ifelse(bars$Date
    %in% signals$date[signals$volume == "CONFIRMED"], "gold", "gray"))
```
This line: The panels below: volume (gold on confirmed days), then the RSI, the MACD and the stochastic.

The boxed part: type = "h" draws a thin bar for each day.

```
87  invisible(dev.off())
```
This line: Closes the picture file. Until then nothing is saved.

The boxed part: dev.off() turns the drawing off.

6 Write the answer

Block 6, lines 88 to 91. Saves the signal days and today's answer, and prints the answer.

```
89 write.csv(signals, "output/signals.csv", row.names = FALSE, quote = FALSE)
```

This line: Saves the table of signal days as output/signals.csv.

The boxed part: row.names = FALSE leaves out the row numbers.

```
90 writeLines(answer, "output/answer.txt")
```

This line: Saves today's answer as output/answer.txt, a fixed file anyone can open.

The boxed part: writeLines writes each piece of text on its own line.

```
91 cat(answer, sep = "\n")
```

This line: Prints the answer in the Console, one line each.

The boxed part: cat prints text as it is.

6. Run it and read the answer

Real runs of the example script, one for each answer. Change the four settings, click Source, read the answer.

1 Buy mode: WAIT

Settings: `mode` is `"buy"` and the stock is KO, the file you downloaded with the end date 30 September. Click **Source**. The Console prints the answer, and the same lines go to `output/answer.txt`.

THE FOUR SETTINGS LINES, AS TYPED

```
3 mode <- "buy"                  # "buy" or "sell"
4 stock <- "KO"                  # which file in the data folder: data/KO.csv
5 bought_price <- 87.29          # sell mode only: the price you paid
6 bought_date <- "2026-08-12"    # sell mode only: the day you bought it
```

R PRINTED

```
WAIT: no buy signal today, 0 of 5 votes say BUY and 3 are needed
Last buy signal: 2026-09-22, enter above 88.97, stop at 87.46
Next one: at least 3 of the 5 votes must say BUY on the same day
```

No buy signal today: the latest day has 0 of the 5 votes on BUY. It also shows the last buy signal in the file and what has to happen for the next one.

2 Buy mode: BUY

The same file for Ford (F), downloaded a day earlier, ends on 29 October, a day with a buy signal.

THE FOUR SETTINGS LINES, AS TYPED

```
3 mode <- "buy"                  # "buy" or "sell"
4 stock <- "F"                   # which file in the data folder: data/F.csv
5 bought_price <- 87.29          # sell mode only: the price you paid
6 bought_date <- "2026-08-12"    # sell mode only: the day you bought it
```

R PRINTED

```
BUY: enter if price trades above 12.45, stop at 11.7
```

Enter if price trades above that day's high, with the stop at its low. Remember that this is a trigger level: the trade happens only if price later goes above it.

3 Sell mode: SELL now

Now change `mode` to `"sell"` and tell the script what you paid and when. Here you bought KO on 12 August.

THE FOUR SETTINGS LINES, AS TYPED

```
3 mode <- "sell"                  # "buy" or "sell"
4 stock <- "KO"                  # which file in the data folder: data/KO.csv
5 bought_price <- 86.71          # sell mode only: the price you paid
6 bought_date <- "2026-08-12"    # sell mode only: the day you bought it
```

R PRINTED

```
SELL now, at about 86.08, -0.7%
Reason: sell signal on 2026-08-17
```

A sell signal appeared after you bought, so the answer is SELL now, at about the last close, with your gain or loss in percent and the reason.

4 Sell mode: HOLD

The KO file with the end date 30 October, and a purchase on 12 October.

THE FOUR SETTINGS LINES, AS TYPED

```
mode <- "sell"                  # "buy" or "sell"
stock <- "KO"                   # which file in the data folder: data/KO.csv
bought_price <- 89.79           # sell mode only: the price you paid
bought_date <- "2026-10-12"     # sell mode only: the day you bought it
```

R PRINTED

```
HOLD: stop at 88.42, +4.8% so far
```

No sell signal and no close below the stop since you bought, so you hold, with your stop and your gain so far.

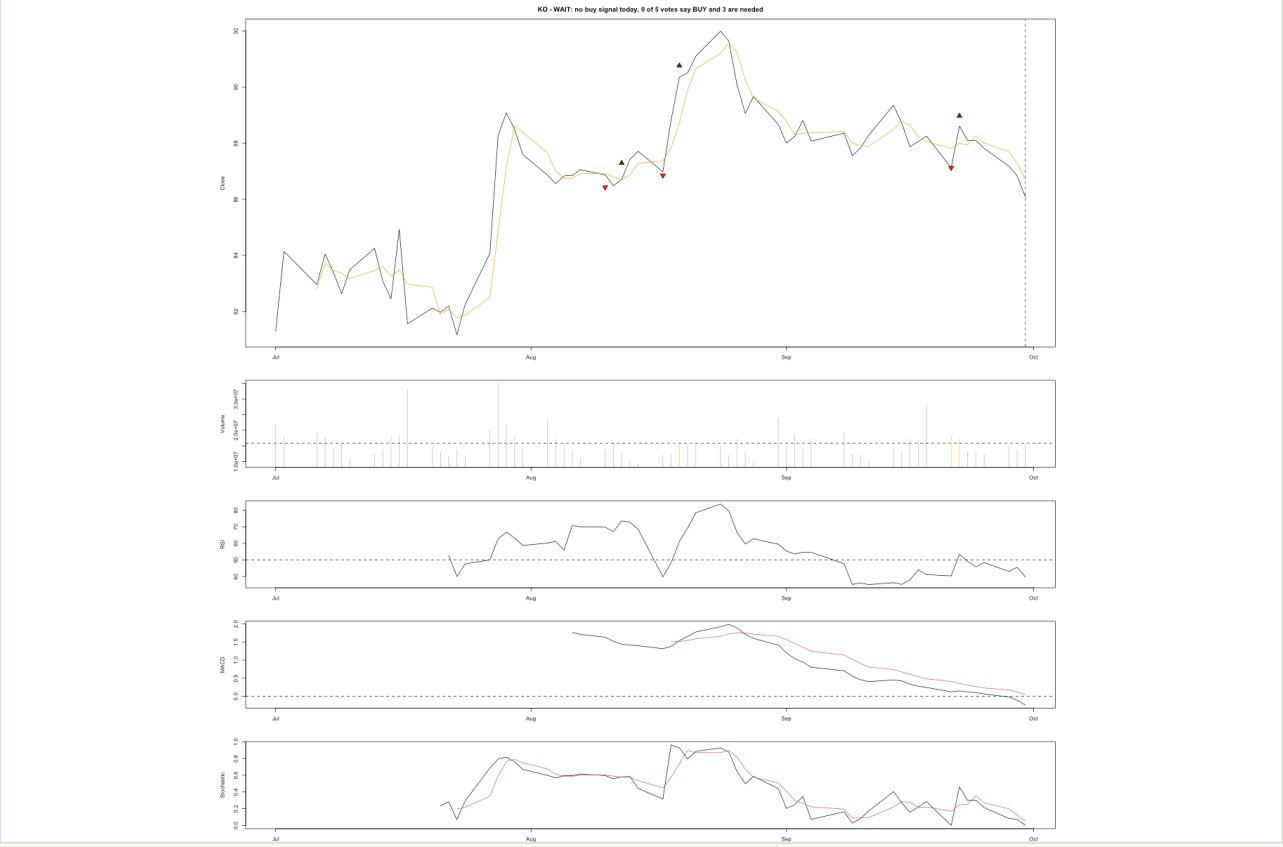
5 Read the table and the chart

`output/signals.csv` has one row for each signal day: the `side`, the trigger level `entry`, the `stop`, what each vote said, `agree`, and `volume`, CONFIRMED when the day's volume is above the file's average.

OUTPUT/SIGNALS.CSV, THE FIRST ROWS (BUY MODE, KO)

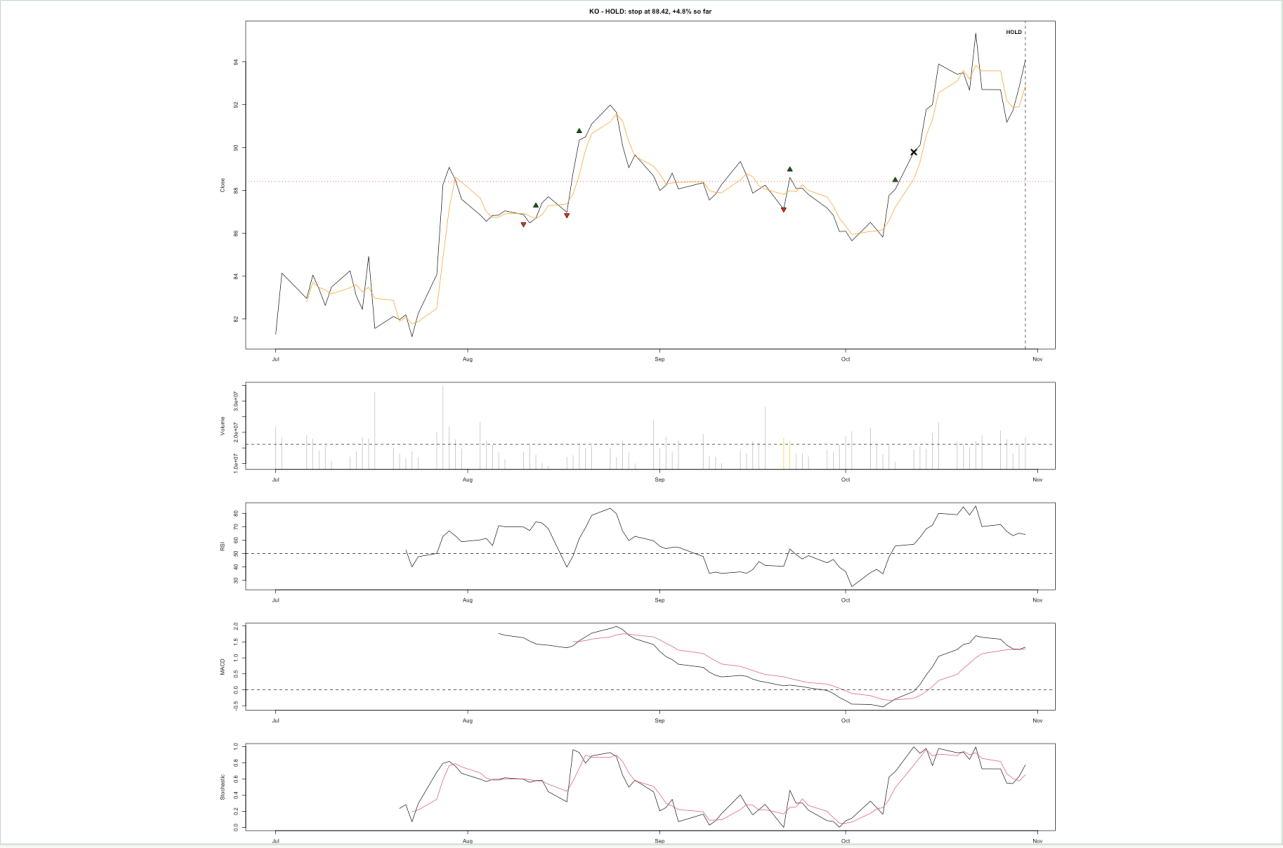
```
ticker,date,side,entry,stop,trend,bar,rsi,macd,stoch,agree,volume
KO,2026-08-10,SELL,86.43,87.53,SELL,SELL,,,SELL,3,NOT
KO,2026-08-12,BUY,87.29,85.66,BUY,BUY,,,BUY,3,NOT
KO,2026-08-17,SELL,86.85,88.02,SELL,SELL,SELL,,,3,NOT
KO,2026-08-19,BUY,90.76,88.59,BUY,,BUY,BUY,,3,CONFIRMED
```

`output/KO.png`: price on top with its orange three bar average. Each triangle is a signal day, green up for a buy and red down for a sell. The dashed line is today, labelled with the answer. In sell mode a cross shows where you bought and a dotted red line is your stop. Below are the volume (gold on CONFIRMED days) and the three oscillators. **The MACD panel is empty at the start:** it needs about 34 days of history, so the first weeks of this 64 day example show none. For your own stock pull about a year.



Buy mode on KO: the answer is WAIT, marked at the right edge.

Drawn by the example script from the Part 2 price file.



Sell mode on KO with the file to 30 October: HOLD, with the cross where you bought and the dotted stop.

Drawn by the example script from the Part 2 price file.

6 Add new data and run again

Download the file again with a later end date (Part 2), replace the one in `data`, and click **Source** again. Nothing else changes. For KO with the file to 30 October, buy mode now answers:

THE FOUR SETTINGS LINES, AS TYPED

```
3 mode <- "buy"                  # "buy" or "sell"
4 stock <- "KO"                  # which file in the data folder: data/KO.csv
5 bought_price <- 87.29          # sell mode only: the price you paid
6 bought_date <- "2026-08-12"    # sell mode only: the day you bought it
```

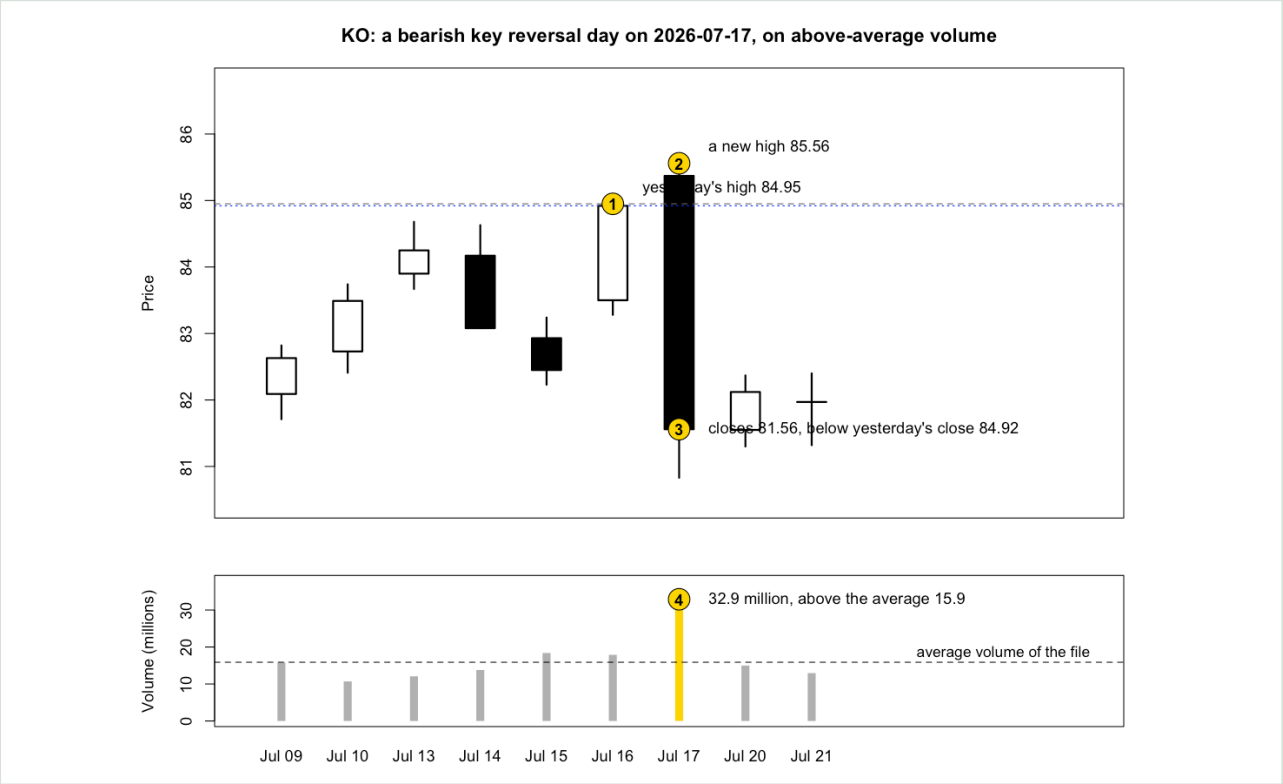
R PRINTED

```
WAIT: no buy signal today, 1 of 5 votes say BUY and 3 are needed
Last buy signal: 2026-10-09, enter above 88.48, stop at 87.39
Next one: at least 3 of the 5 votes must say BUY on the same day
```

7. Why the signals are where they are

Real signals from the example tables, drawn for the days around them. The script prints numbers only.

A price bar on volume. The bar vote is the key reversal day (slides 123 and 126). In the picture, mark 1 is yesterday's high, mark 2 is today's new high, and mark 3 is the close, below yesterday's close: the buyers pushed price up and could not hold it, so the vote is SELL. Mark 4 is the volume: above the average of the file, so the day is CONFIRMED (slide 128). Volume does not vote. It says how much weight the move carried.

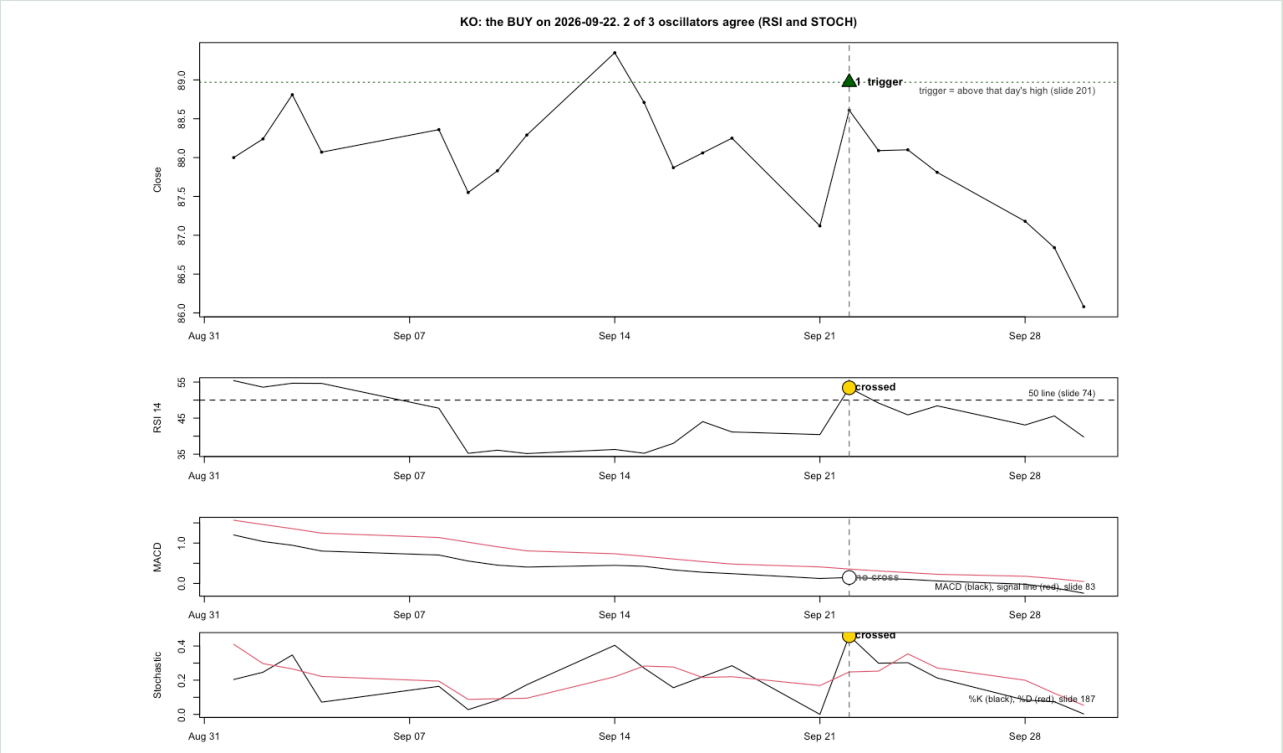


KO, 2026-07-17: a key reversal day, vote SELL, on 32.9 million shares against an average of 15.9.

Drawn by build/r/explain_bar.R from the example price file.

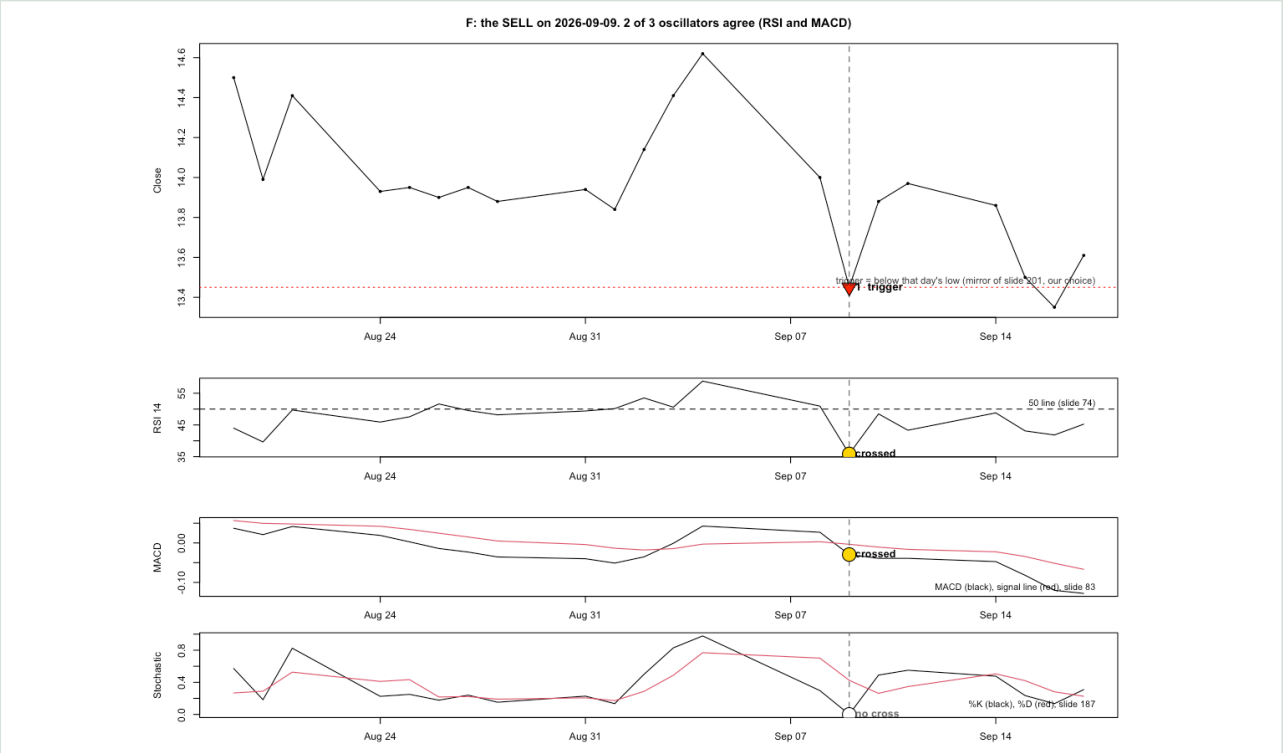
An oscillator buy. On the day the RSI, the MACD or the stochastic crosses upward, that oscillator votes BUY. The trigger is above that day's high (slide 201): the trade happens only if price later goes above it. Gold circles mark the oscillators that crossed and white ones those that did not.

A sell is the mirror image: an oscillator crosses downward. The slides give no sell trigger, so the trigger is below the signal day's low: the mirror of Lim's buy trigger on slide 201, our choice, not Lim's rule. The trade happens only if price later falls below it.



KO, 2026-09-22: a BUY where RSI and STOCH cross. Trigger 88.97.

Drawn by build/r/explain_osc.R from the example price files.



F, 2026-09-09: a SELL where RSI and MACD cross. Trigger 13.45.

Drawn by build/r/explain_osc.R from the example price files.

> AGREEING IS NOT PROOF
>
> Votes agreeing sounds like witnesses agreeing. It is not. The RSI, the MACD and the stochastic are all built from the same closing prices, so they move together: on the example data the RSI and the stochastic have a correlation of 0.73 and the RSI and MACD 0.80. Lim calls this **multicollinearity** and warns that it creates a **false consensus** and a false sense of confidence (slide 78). His cure (slide 81) is to use data other than price, such as volume, which is why the volume check is separate. A signal is only a statement of what a rule says that day, not a forecast.

8. The contract: what every script must do

Your script is marked by running it on a file you have never seen, in buy mode and in sell mode, so everybody follows this exactly.

Folders and files

WHAT	RULE
Where it runs	From the folder that holds the script, which holds a folder called `data`.
The settings	The first lines of your script are these four, in this order, with exactly these names, each on its own line, so the marker can change them: `mode <- "buy"`, `stock <- "KO"`, `bought_price <- 87.29`, `bought_date <- "2026-08-12"`. `mode` is `"buy"` or `"sell"`. A comment after a value is fine.
Input	`data/<stock>.csv`, at least 60 daily rows, oldest first. Columns `Date, Open, High, Low, Close, Volume` exactly. `Date` may be `2026-07-02`, `2026-07-02 16:00:00` or `7/2/2026 16:00:00` (Google Sheets).
Output	`output/signals.csv`, `output/answer.txt` and `output/<stock>.png`: price on top, volume and the three oscillators below, signal days marked, today's answer marked.
Packages	Base R and `TTR`. Nothing else, and no `install.packages` inside the script.
Time	Under 30 seconds for each run.

The columns of output/signals.csv, in this order

COLUMN	MEANING	EXAMPLE
`ticker`	The stock: your `stock` setting	`KO`
`date`	The signal day, `YYYY-MM-DD`	`2026-09-22`
`side`	`BUY` or `SELL`: the side your rule says agrees enough that day	`BUY`
`entry`	The trigger level, by your rule. The example: a buy is the signal day's high, a sell its low	`88.97`
`stop`	The stop, by your rule. The example: a buy's is the signal day's low, a sell's its high	`87.46`
`trend, bar, rsi, macd, stoch`	What each vote said that day: `BUY`, `SELL` or empty. These five are the example's. Your script has one column per vote of its own, named to match	`BUY`
`agree`	How many of your votes gave the `side`	`3`
`volume`	`CONFIRMED` if that day's volume is above the file's average, else `NOT`	`CONFIRMED`

One row for each signal day, oldest first. Zero rows is fine: the file still has its header.

output/answer.txt: what the marker reads

MODE AND ANSWER	THE FIRST LINE STARTS WITH, AND CONTAINS
buy mode, BUY	`BUY` and the two levels: the level to enter above (today's high) and the stop (today's low).
buy mode, WAIT	`WAIT`. Further lines give the last buy signal in the file: its date, its high and its low.
sell mode, SELL	`SELL` and the last close and your gain or loss in percent.
sell mode, HOLD	`HOLD` and your stop and your gain or loss in percent.

Use your own words if you like, but the first word and these numbers must be in the file. The example script is the model.

9. Your task

Your own program, on your own stock: the rules are yours to write, not the example's.

Your job is not to copy the example script. Your job is to write your own program. Choose your own stock (a US stock, downloaded as in Part 2, about a year of daily rows) and write your own rules for at least **two** of these three: (a) which indicators vote, or how a vote is decided, (b) how many votes must agree for a signal day, (c) where the stop goes. State your rules in comments at the top of your script, plainly, so the marker can read them. Everything else stays fixed so the marker can run every script the same way: the same four settings lines at the top (`mode`, `stock`, `bought_price`, `bought_date`), the same answer words (BUY or WAIT in buy mode, SELL now or HOLD in sell mode, with the numbers), and the three output files of Part 8. Test it yourself: run it in buy mode, then change `mode` to `"sell"`, type a price and a day you bought, and run it again. Then draw a **flowchart** of how it works, and mark on it where your program differs from the example.

A COPY IS WORTH ZERO FOR THE SCRIPT PART

A script that is the example with only the stock changed, or that matches the example line for line, gets zero for the 60 script points, even if it runs and writes the right files. Changing one of the three things above is not enough either: you need at least two.

DUE DATE

Sunday, 15 November 2026, 11:59 PM.

THIS ALSO COUNTS AS YOUR CHAPTER 8 QUIZ AND YOUR FINAL QUIZ

There will be no reschedules for this activity: late or missing work is not re-opened. This is too hard to check case by case, so there is no exception.

Name your files exactly

FILE	NAME
Your script	`FirstAutomation_Surname_Firstname.R`
Flowchart, PDF (preferred)	`FirstAutomation_Surname_Firstname_Flowchart.pdf`
Or the .bpmn file	`FirstAutomation_Surname_Firstname_Flowchart.bpmn`
Or a Markdown file	`FirstAutomation_Surname_Firstname_Flowchart.md`

Two files, nothing else. No price files or output: they are rebuilt from your script.

How to submit

1. In your own Google Drive, make a folder named FirstAutomation_Surname_Firstname (your own surname and first name).
2. Put your two files in it, named as in the table above.
3. Share the folder with benjiesotelo@gmail.com as Viewer before the deadline.
4. Do not change the files after the deadline: the time each file was last changed is checked.

Before you submit, check

1. Your comments at the top state which two (or more) of the three things you changed, plainly.
2. Put your script in a clean folder with a `data` folder holding your stock's file, and run it in buy mode and again in sell mode, changing only the four settings lines.
3. `output` has `signals.csv` with the columns of Part 8, `answer.txt` and a `.png` named after your stock.
4. Delete `output` and run again. It must come back.
5. Open your flowchart next to your script. Every shape has lines behind it, and it marks where you differ from the example.

> THE MARKER RUNS YOUR SCRIPT ON FILES YOU HAVE NOT SEEN
>
> A file you have not seen, in the layout of Part 8, run in buy mode and in sell mode with settings the marker chooses. Some files write dates as `2026-07-02`, some as `7/2/2026 16:00:00`. A script that only works on the example file, or that needs more than the four settings lines changed, will not pass. The answers it writes are checked against your own stated rules, not against the example's.

10. Using AI

Allowed, with one condition.

You may use AI to help write your script. The condition is that **you can properly visualise and explain it in your flowchart**. The flowchart is where you show that you understand the script, so:

- The flowchart must match **the script you actually submit**, not an earlier version and not what the AI described.
- Each task must say, in your own words, what that part of the code does, and where the rule has a number, which slide it comes from.
- If you cannot explain a part of the script, do not submit it. Ask for a simpler version you can explain, but it still has to be your own program on your own rules (Part 9), not the example with the stock changed.

A script that runs perfectly with a flowchart that does not match it loses the flowchart marks. A modest script with a flowchart that matches it and explains every step does well. None of this is a reason to hand in the example itself: that scores zero for the script part, however good the flowchart.

11. The marks

100 points. 60 are checked by running your script and reading your stated rules, and 40 are judged from your flowchart.

Checked by running your script

WHAT	MARKS	HOW
The script runs	10	In buy mode and in sell mode, on a file you have never seen, with no change except the four settings lines and no installs, in under 30 seconds each.
It writes the files the contract names	10	`output/signals.csv` with the required columns (4), `output/answer.txt` (3), and `output/<stock>.png` (3).
It finds the signal days your own rules give	15	The date and side of every signal day match what you stated in your comments, with none missing and none extra.
The buy mode answer follows your own rules	10	BUY or WAIT, and the numbers in the answer, follow what you stated, not the example's rules.
The sell mode answers follow your own rules	15	SELL now or HOLD, and the numbers in the answer, follow what you stated, for two different bought dates.
Subtotal	**60**	

Right or wrong against the rules you stated in your comments, not against the example's rules. A script that is the example with only the stock changed, or that matches it line for line, scores zero here, whatever it prints.

Judged, with the course rubric on the next pages

WHAT	MARKS
The flowchart is correct BPMN	10
The flowchart matches your script	15
The flowchart explains each step	15
Subtotal	**40**

Includes whether your flowchart shows where your program differs from the example.

NOT MARKED

How your code looks. Neat or messy, short or long: not marked. Only whether it works, whether your comments state your own rules plainly enough to check, and whether the flowchart explains it.

YOUR FLOWCHART IS MARKED ON ITS OWN PAGES, AND THE SIGNALS ARE NOT A FORECAST

The script reports what a rule says at a point in the past. Nothing here is marked on whether a signal would have made money.

How a written answer is marked

For the 40 points that a record cannot check. The same rubric marks every written answer in this course.

> READ THIS FIRST
>
> **We are marking whether your answer is grounded and whether it follows, not how long it is.** A long answer that wanders earns less than two precise sentences.

This rubric is for written answers that have no single correct response. Anything that can be checked against the record, a price, a date, a count, is simply right or wrong, and no rubric is needed for it.

What earns the marks

- **Grounded.** Your answer points at something specific and checkable on your own chart: a price, a date, a shape. Not "the market looks bullish", but "the trough on this date, at this price, is higher than the one before it".
- **Connected.** Your conclusion follows from what you pointed at. It is possible to name three real things and then conclude something that has nothing to do with them.

> HONEST: NAMED, NEVER MARKED
>
> A strong answer also names what argues against it. This is what the best answers do, and it is worth learning to do. **No marks are lost for leaving it out.**

The levels

LEVEL	SHARE OF THE MARKS	WHAT IT LOOKS LIKE
Full	100 percent	Grounded and connected.
Partial	75 percent	One of the two is weak: the evidence is vague, or the conclusion does not quite follow from what was named.
Minimal	50 percent	There is a reason, but it is general rather than taken from your own chart.
None	0 percent	A conclusion with no reasoning, or nothing written.

Percentages rather than marks, so the same rubric marks a question worth four points or forty.

The written answers on this sheet, level by level

ANSWER	OUT OF	FULL, 100%	PARTIAL, 75%	MINIMAL, 50%	NONE, 0%
The flowchart is correct BPMN	10	10	7.5	5	0
The flowchart matches your script	15	15	11.25	7.5	0
The flowchart explains each step	15	15	11.25	7.5	0

Every figure in this table is the level's percentage of what the answer is worth, so it cannot disagree with the levels above.

What the rubric looks for in each of them

ANSWER	GROUNDED HERE MEANS	CONNECTED HERE MEANS
The flowchart is correct BPMN	Points at something checkable in your diagram: a start and an end event, a labelled `yes` and `no` leaving every gateway, an arrow back for every loop.	The shapes are used for what they mean: a diamond is a question, a rectangle is an action, and no path stops without an end event.
The flowchart matches your script	Points at the line of your script each shape stands for, and at every loop and decision in the code having a shape, with the shapes that differ from the example marked as such.	Following the arrows reproduces what running the script does, in the same order, with no shape for code that is not there, and a reader can see where your rules depart from the example's.
The flowchart explains each step	Each task says what its lines do, with the actual column, price or file the code uses, and names the slide where the rule has a price.	The explanation tells the reader why that step is there, not only what it is called.

One question, answered at every level

Here is task 9 of the example flowchart explained five ways, one at each level, and two at Partial because there are two ways to be partial. They are written on the example, not on yours.

LEVEL	THE ANSWER	WHY IT SITS THERE
Full, 100%	Task 9, Record the signal day: it keeps only the days when 3 or more of the five votes agree, so a day with two BUY votes is skipped. A BUY day's trigger is its High, as slide 201 says, a SELL day's is its Low, the mirror, and `agree` counts the votes on that side.	Names what the code keeps, a real consequence (days skipped), the exact price rule and the slide.
Partial, 75%	Task 9, Record the signal day: it keeps the days with a signal and writes the trigger using the slide 201 rule.	Right idea and a slide, but it never says how many votes make a signal or what `agree` counts, so it cannot be checked.
Partial, 75%	Task 9, Record the signal day: it keeps the days with 3 or more votes, because three votes agreeing means price will rise.	Checkable action, but the reason does not follow: the code does not predict anything.
Minimal, 50%	Task 9, Record the signal day: this is where the program decides which days matter.	A reason, but general: it could describe any task.
None, 0%	Task 9, Record the signal day: signals.	A label with no reasoning.

> WHAT THE BEST ANSWERS ADD
>
> A strong flowchart also says where its script is not Lim's rule: "the slides give no sell trigger, so this script uses the low, the mirror of the buy trigger." It is never marked, and it is the habit that makes a rule you automated one you can trust.